

Mentoring trajectories in an evolving agile workplace

Shreya Kumar
Michigan Technological
University
1400 Townsend Drive
Houghton, Michigan USA
ShreyaK@mtu.edu

Charles Wallace
Michigan Technological
University
1400 Townsend Drive
Houghton, Michigan USA
wallace@mtu.edu

Michael Young
ThermoAnalytics Inc.
23440 Airpark Blvd
Calumet, Michigan USA
mfy@thermoanalytics.com

ABSTRACT

Agile approaches to software development offer flexibility and autonomy to developers, while demanding discipline and attentiveness. At its best, agile constitutes an idealized vision of Wenger's Community of Practice: one where the essence of the software practice is freely negotiated by participants following self-determined trajectories of identity within the community. While agile frameworks prescribe ways of doing, in reality these codified practices must adapt to a real workplace. Nowhere is this more plainly seen than in the onboarding stage of new employees: this is the point where newcomers establish an identity and establish means of participating within the development community.

We report on the evolving nature of onboarding and mentoring at a mature software development firm that has practiced Scrum for a decade. As the firm has scaled up to an international purveyor of specialized engineering software and services, the way in which new employees are brought into the community of practice has changed profoundly. We view this history through a series of lenses: those of workplace participants entering the firm at different stages of the firm's history. We follow the history of the onboarding process through the perspectives of several employees, and we focus on the trajectories of certain key employees as their roles change from novice to mentor.

Categories and Subject Descriptors

D.2.9 [Software Engineering]: Management—*Programming teams, Software process models*

Keywords

Software engineering, ethnography, participant observation, communities of practice, mentoring

1. INTRODUCTION

Agile involves adaptation. While frameworks like Scrum, Kanban, Crystal Methods and Extreme Programming offer

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICSE '16 Companion, May 14 - 22, 2016, Austin, TX, USA

© 2016 Copyright held by the owner/author(s). Publication rights licensed to ACM. ISBN 978-1-4503-4205-6/16/05...\$15.00

DOI: <http://dx.doi.org/10.1145/2889160.2889236>

idealized settings, in practice they must match the reality of the workplace. Within a software development environment, there will be varying levels of buy-in to agile ideals, disagreement (overt or implicit) about the meaning of agile principles and practices, varying levels of experience, and varying goals and identities. As participants enter and leave a development team, as their roles and perspectives change over time, and as realities of the marketplace and workplace impinge on the team, the meaning of “agile” within the community is under constant evolution.

The process of *onboarding* is a crucial factor in this evolutionary process. Through onboarding, newcomers establish an identity and means of learning within the workplace. Understanding how this process works is a crucial concern for software enterprises, both for establishing new employees as capable, confident participants, and for making the best use of the time of experienced employees as they mentor the new employees.

TAI, the subject of our study, is a firm of over 60 employees headquartered in Michigan's Upper Peninsula, with four facilities in the United States and Europe. TAI provides software and services for thermal modeling in a variety of engineering fields, including automotive, aerospace, and defense applications. For the past decade, TAI has used Scrum and other agile methods as a framework for its software development practice.

2. TAI: AN EVOLVING PRACTICE

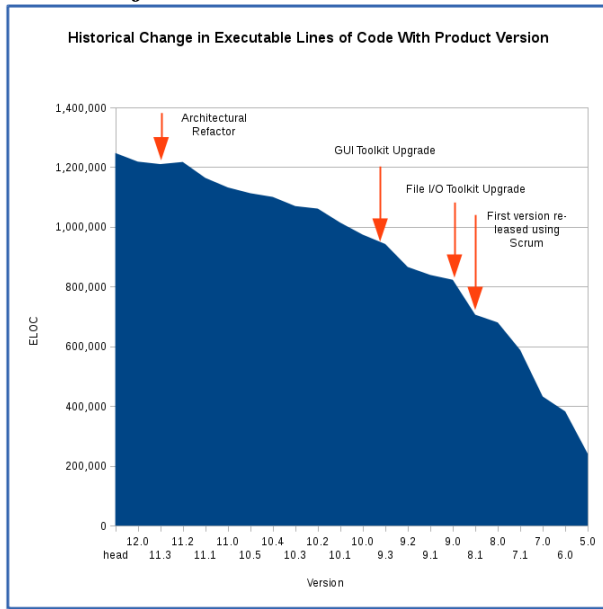
TAI was formed in 1996 as a spinoff from Michigan Technological University, in Michigan's Upper Peninsula. The domain area of the firm — thermal modeling for engineering applications — has remained constant through its history. TAI's identity, however, has changed as it has moved grown in scale and scope. Market forces have shifted the product vision and the focus of activity. Successful products have led to an increase in scale and therefore a change in makeup and identity of the employees. (Figure 1 gives an indication of the increased complexity of the code base.) This in turn has led to adoption of new practices and approaches to process.

The rough timeline below gives a sense of the interplay between these forces. **Marcus**, a long-time employee and current leader of the software development team, is our primary source for this history.¹

1996–1998: The firm operates in a way similar to its roots as an academic research group. Funding is primarily from

¹Names of individuals in the paper are pseudonymous. Each initial reference to a participant in our study is given in **bold**.

Figure 1: Change in lines of code with version number and major events



government contracts, leveraging SBIR (Small Business Innovation Research) awards. Researchers work essentially in an autonomous fashion on the shared code base, interacting only occasionally through ad hoc code reviews.

1998–2002: Corporate funding leads the firm in a more targeted direction. With a specific need for a commercial radiation transfer analysis solution, the first commercial application is born. At this stage, TAI still identifies as an engineering firm, not a software firm. Developers have backgrounds in science and mechanical engineering rather than software engineering. Basic software development mechanisms (e.g. revision control, bug tracking, formal code reviews) are brought in, prompted by commercial development. Development, however, still proceeds in a largely decoupled fashion among researcher/developers. There is still no explicit acknowledgement that a principled software process is needed, though individuals are starting to recognize this.

2003–2007: The firm moves to an international market. It positions itself as providing engineering services for hire: TAI employees are hired to model customer engineering scenarios, using the internally developed and used software product. This period is marked by an increase in development, with more people brought on due to external funding. An internship program is started in 2003, and dedicated software developers (not engineers or researchers) are added. Three of the eight developers have CS degrees. Mentoring of new developers is done in an implicit fashion by “pioneers” who did the original exploration of the space. With the increase in scale, addressing issues of personality fit and matching skills to needs proves more difficult. With a new constituency of developers not steeped in the history of the company or the research, further guidance is needed. In 2006 Scrum is adopted as a development framework.

2008–2010: Customer demand exceeds the engineering service model, and the firm pivots toward providing a consumer product, to be offered side by side with service. User expe-

rience issues take on greater importance. In 2009 a formal onboarding process is introduced for development interns.

2011–present: As scale increases, special needs and cross-cutting concerns are identified. The time scale implicit in Scrum is considered too long for basic maintenance — “trying to tango to dubstep”, in the words of Marcus — so in 2013, a team dedicated to maintenance is formed, using Kanban with weekly cycles. A challenge emerges: how to balance the time commitments of the pioneers between development and mentoring? These team members have deep knowledge of the code base and can make valuable contributions to the code, yet that knowledge also makes them the best choice for mentoring newcomers. New employees must now rely on non-pioneers for their onboarding experience.

The engineering services and research department members or the product owners often serve as proxy clients. Depending on the strength of the software development department, big stories like feature requests and major bug changes are taken up by the 4-8 person teams following Scrum, where the team commits to a set of stories for a 3 week sprint cycle. Depending on the size of the story, an appropriate number of developers self select to work on a story.

With 20 developers now on the team, Marcus finds that he cannot handle all as direct reports. In his account of the early days, “you had to know everything about everything”. Now with 2 million lines of code, this is not feasible. He notes a struggle with Scrum: while avoidance of narrow silos is advantageous, how general do you have to be? For real employee satisfaction, there must be a certain catering to personal interests and a vision of ongoing professional development.

The decision is made to set up skill teams with domain areas of expertise. Organizational patterns using the Spotify model [7] are adopted. Vertically, squads work as cross functional teams on future feature sets. Horizontally, chapters are formed on the basis of cross-cutting subject matter expertise. For instance, the physics chapter includes more science and engineering focused personnel, interested in numerical methods; the presentation chapter deals with user facing issues and technical documentation; and the productivity group discusses process issues: testing, DevOps and agile. Each chapter has a leader with traditional management responsibilities. Chapter leads determine who (among those who have expressed interest and commitment) can join a chapter. Marcus’s mentoring process for chapter members is a work in progress.

Marcus relates his account of the current onboarding process: small groups (one on one or two on one) work on a specific project for 2–4 weeks, writing a utility program or an exploratory feature, or working on maintenance of the product line, fixing defects. Onboarding is experiential learning, with indoctrination into process woven into the technical work. During the first week, newcomers read up on Scrum and coding standards. With a customer proxy and their mentor, they define stories, then requirements, acceptance criteria, design solution, and implement/test. Newcomers engage in pair programming, two screens, one for code writer, other for test writer. Reflecting on his own mentor role, Marcus describes a sensitivity to personality and skill level, and deliberately “driving” less as he senses greater confidence on the part of the learner. Mentorship is a rotating responsibility: mentors are called off of their other duties for extended periods: multiple weeks or months. Marcus assesses poten-

tial mentors for fit to the work: “we wouldn’t do that to every person”.

3. COMMUNITY OF PRACTICE

TAI’s history of evolution and reinvention is highly representative of the software industry. It is important to study the essence of how these dynamic software development communities manage and sustain themselves. Some of the main challenges that software developers or software engineers face in industry are related to their lack of expertise in the “soft skills” of communication and teamwork, as described in the works of Begel [1] and Begel and Simon [2] on novice software engineers. In addition, the research by Hall *et al.* on employee turnover in software engineering projects [5] describes the negative impact on the motivation in and success of a project. Zanetti [12] talks about the influence of social factors in turnover, where employee turnover is a reality of the typical software project experience.

We interpret participation in software projects as a negotiation of meaning, as defined by Wenger [14] in his concept of Community of Practice. Wenger describes communities of practice as “groups of people who share a concern or a passion for something they do and learn how to do it better as they interact regularly”. He views the *meaning* of the community and its practices as essentially linked with *identity*. Those who identify as community members contribute to the shared negotiation of meaning, and participation in the negotiation of meaning reinforces identity.

Considering software development communities as communities of practice is not a novel idea. Paasivaara and Lassenius [11] study “large, distributed agile software development” as a community of practice at Ericsson. Their work describes a large software company using the community of practice concept in the design of their processes. It is important to note that in Wenger’s work, the term Community of Practice is descriptive rather than prescriptive. Even a software development community, like TAI, that does not identify itself with the term Community of Practice is still a community of practice, where members join, leave and work together to share and shape knowledge and negotiate their role and the meaning of the system they are building.

3.1 Evolving meaning, evolving identity

In computing disciplines it may seem unusual to think of meaning as negotiated, as the mathematical roots of code suggest a conception of meaning as precise and unambiguous. However, in software engineering, code is only a manifestation of the developers’ or designers’ mental model of the system, which in turn is a representation of users’ and clients’ requirements. To arrive at the proper implementation of these requirements, interaction is needed among multiple stakeholders with different roles and involvement in the software development community or that particular software project community, and the “meaning” is negotiated through different and possibly conflicting interpretations of the requirements and design specifications. In addition, a community member’s role, influence and contribution in a software development community of practice may also be negotiated over time as members join, leave and work in different capacities over time.

At TAI, the meaning of onboarding and mentoring has evolved. This is partly due to market demands, as TAI’s software becomes a full fledged consumer product, and re-

searchers no longer have the luxury of total independence they once had. It is also due to scale, as new developers outside the area of expertise need to come up to speed. The evolving meaning is partly articulated through explicit changes from management, but as we shall see, it also occurs in a tacit manner, as newcomers improvise new activities and relationships to get what they need.

Examples of negotiated meaning in the TAI case (which we discuss at length later):

- The maintenance team becomes a pre-release team and increases in size from three to eight, with a switch from Scrum to Kanban. The larger team, is a mixture of newbies and more experienced employees. How does the new way of doing maintenance get defined?
- Within the maintenance team, a discussion emerges concerning the template for closing a bug. What is the appropriate level of rigor for such a template?
- In the maintenance retrospective, it emerges that some bugs require work that exceeds the bounds of Kanban. Should bugs be sized as stories? Is the Product Owner the appropriate arbiter for bug size?

Examples of evolving identity at TAI:

- Pioneers find themselves in a difficult situation: with their intimate knowledge of physics and code, they are extraordinarily productive in terms of story completion. But ultimately, they have more value in serving as mentors and educating others. In §5.1 we focus on one pioneer’s journey from researcher to programmer to implicit then explicit enabler.
- Similarly, employees who were onboarded in an earlier era, receiving their mentoring from pioneers, are now put into role of mentor. In §5.2, one early onboarder who experienced different mentoring styles as a protégé, reflects on that difference in producing her own style when she becomes mentor.

3.2 Participation and reification

Wenger describes the concept of *participation* as social engagement in a community as a member, where this experience is both personal and social. It can manifest itself as thinking, being, doing, talking, feeling of social relations and emotions. Our participation in a community shapes our experience of the world and in turn also shapes that community. Participation can take many forms and may be formally acknowledged or occur informally or even subconsciously. One may not perceive their participation as contributing anything to the community, but their mere participation helps shape that community in some way.

Wenger’s notion of *reification* refers to either concrete products or conceptual products as well as the process of producing the products of participation. Reification can occur by making, declaring, recasting, perceiving, articulating, designating, representing, using, *etc.*

Participation and reification exist in a duality, where they complement each other as a balance. For example, a developer working on her first story engages in a social process that is reified as working code. This is a natural part of the onboarding process at TAI: new employees are immediately put on to new tasks away from the main production line, or

given maintenance tasks, and they are expected to commit code and report a bug within days of joining. Newcomers report that this allows movement from observer to contributor: the act of committing shows you how to participate.

In software development communities of practice, communication has a profound effect on both meaning and identity. Members are continually choosing between multiple communication options. The right choices at the right time can help bring participants in and facilitate convergence of meaning, while the wrong choices can marginalize participants' identity and confound the negotiation. Although there are many norms and standards of communication that help, members still have to be creative in their context sensitive communication choices on a daily basis. Agile software development works within a framework of communication rituals which also indicates the importance of communication to the software process overall. The rituals and artifacts are not meant to be a manual to resolve all communication issues, they are a loose framework to facilitate better and more creative communication.

Wenger describes how identity is defined by participation as well as non-participation and how different communities may have different means to reify identity. Some communities may opt for formal and structured reification methods and some might employ more implicit means. Participants in a community also determine their identity by choosing different types of trajectories into the community. Participants thereby contribute to the definition of their identity by choosing a combination of different trajectories, levels of participation and negotiating the meaning of their identity through their quantity and type of contribution.

3.3 Mentoring

In collaborative creative endeavors like software development, newcomers must be brought up to speed not only on matters of fact but on deeper issues of rationale and motivation. The concept of mentor — the experienced guide, conveying knowledge and “know-how” to the protégé — is a time-honored tradition in management. Whether through established, codified practices (*e.g.*, explicit mentoring initiatives by professional engineering organizations [10]) or the more implicit processes captured in legitimate peripheral participation [9], mentors provide instruction, counseling and interaction to impart understanding in a way that “reading the manual” (or the source code) cannot.

Software development, however, occupies a unique position in this space, due to its innately fluid and fast-changing nature. Software teams are formed and reformed at a rapid pace, in response to evolving requirements, business alliances, and personnel changes. Moreover, the flexibility afforded by software development, exemplified most vividly by open-source projects [15], allows theoretically limitless numbers of collaborators, problematizing the notion of team altogether. In this context, the concept of mentor must be expanded beyond its customary definition. Mentoring relationships may be ad hoc and transitory, with little or no clear delineation between those eligible for mentor status and those seeking mentorship.

Several scholars have identified communication as a central aspect of the mentoring process. Beyond the “simple exchange of information and accomplishment of ability” which is the primary goal of mentoring, Kalbfleisch [6] likens the process of establishing a mentoring relationship to “the ini-

tiation of friendships and love relationships in terms of communicating appropriate relational expectations”. Buell [3] expands on this idea by categorizing mentoring relationships in terms of cloning, nurturing, friendship and apprenticeship, and noting the importance of “turning points” where the nature of the mentoring relationship changes.

4. METHODOLOGY

One member of our research team worked as a software developer in the industry software development team for eight months, studying the team as a participant observer and conducting occasional interviews with key team members, especially those that work closely and have varying degrees of experience and serve different roles. We use a combination of personal observations and interview data to uncover themes in communication and mentoring strategies.

4.1 Onboarding

Our participant observer **Natalie** started working at the company as a software developer intern in January of 2015. It was the first day for another newcomer, **Nathan**. They were given a tour of the offices and were introduced to other software development team members. The new employees attended an IT introduction, getting access to their computers, software and the common code base, and then were given their team assignments. Natalie and Nathan were placed in the “maintenance” team, following Kanban, along with one existing team member.

After one and a half months of working on maintenance as a three-person team, three to five new members previously working on the development team were added to the maintenance team and the work was re-prioritized to bug squash for a major version product release. This team was rebranded the “pre-release” team. In a larger team, a more authentic Kanban practice was possible.

After working in the maintenance team for a total of about three months, Natalie and Nathan were assigned to one of the Scrum development teams, where many of the other pre-release team members were also assigned.

4.2 Data Collection

Our study is based on three types of data:

- the participant observer’s daily communication activity data for questions like how often do developers work together or alone and for what kind of work;
- interviews of selected team members to capture their experiences and journeys;
- personal reflections of our participant observer about the communication and teaming practices regarding onboarding, role and knowledge sharing.

The participant observer tracked daily communication data through an electronic calendar, adding entries for the time blocks that she worked in different modes: working alone on a task, pairing with another developer for the same task; attending the daily stand up meeting; participating in a design discussion with two other members. This calendar data was then converted into a format more appropriate for analysis.

For the interview data, our approach was adaptive. We spent some time just observing how different people across the teams interacted with each other and through discussions with the gatekeeper, who knew the history and journey

of all the team members, identified a few select developers to interview. The interview questions were specifically designed to discover the journey of the interviewee through this community of practice. Interviewees were asked to describe what they remember of their onboarding experience, what they found effective in allowing them to feel like they are part of the team, what different roles did they play, how did their roles change, who helped or guided them through transitions and how, etc.

4.3 Selecting and scheduling interviews

We intentionally picked developers with different levels of experience, ranging from someone who has been a developer for decades, to developers who have a few years of experience as part of this company, to developers who are in the process of being inducted into the community during our observation window. The interviews were conducted with some planned seed questions and more probing specific questions constructed in situ, meant to guide the interviewee to elaborate on their story. Not all the potential candidates identified through discussions with the stakeholders were included in the study. Some candidates were willing to participate in the study but were largely unavailable due to time conflicts and work related travel and vacation time.

The culture of the company encourages its employees to not go beyond a 40-hour work week and the employees typically have to log all of their time spent at work as associated with whatever category of work they were involved in, even if the time spent on a particular task in a category was small. This makes most employees very conscious of how they spend their time, and most team members try to be productively engaged while at work. Although this leads to a healthier and happier work environment, it was sometimes an obstacle for finding time with participants to conduct interviews, as the interview work was not productive for the goals of the company. Most of the developers included in the study were also difficult to get a hold of for even smaller interviews, as everyone who works in the company tends to work hard all day and then go home when a day's work is done. Participating in interviews meant that participants were sacrificing their time after a full day of work when they would rather be heading home.

It was also unwise and unkind to schedule interviews at the end of some days when the work was more taxing than usual, like around the time of a major product release. These were times when most folks would go home tired and times when their current and uncommon state of mind may cause bias in their answers. Most interviews were conducted towards the end of the day, on a semi-planned basis, where most participants would be asked earlier that day if they had time for an interview towards the end of the day. This would allow participants to plan their day accordingly and be mentally prepared for a delay in getting home. Some interviews were conducted outside the office over breakfast during the week-end, for lack of availability during office hours. Interviews ranged from fifteen minutes to over an hour, typically lasting about half an hour.

For this paper, we use the team members' interviews and observations and reflections of the participant researcher.

5. TRAJECTORIES IN ONBOARDING AND MENTORING

We study themes of changes in process over time to accommodate the growing team or the changed focus. We have chosen a diversity of perspectives: the pioneer, the "early on-boarder" (mentored by pioneers), and the "next generation" (mentored by non-pioneers).

5.1 Pioneer's journey

Birth of a pioneer. Alex was recruited by Leon to work at the company as an intern while he was getting his mechanical engineering degree. The company was very small then. Only the core founding members and a few interns worked in the company.

"I was working from 3 to 6 PM one day, the next day not at all. It was a part time job at that time where you are kind of setting your own place."

Alex was given the task of making the GUI. He often worked alone for a long time. *"I was just given some tasks, go off and figure out things and GUI related tasks. I was basically told that go off and do this work. I had access to the stack of X11 reference books."*

Alex then finished his degree and internship and went to work in the mechanical engineering industry for two years. Then he decided to come back and work for the company full time. *"There were four software developers and essentially we were given a task, which was more like a project. Back then I was given the task of starting over to implement the (base) application. It wasn't so much a task as it was a project . . . It was kinda figure out what you want to do, talk it over with the three others, and then you work on it."*

This is the mode the development team operated in for "a large number of years . . . We would meet in all hands meet, and report that we were 90% done. 90% till you are all done, whether you were done that much or not."

Alex would typically work alone with little input from others, and rare communication within the "team". Other programmers would also work separately. Hal was in charge of the software development team, and the process *"was kinda waterfall but still ad hoc."*

Building a community. Alex recollects how collaborative design was not done with the granularity that the team now adopts. *"We did some high level design and made tasks that were bigger than the high level stories we do now . . . It was a team, but it was a loose team and you get a lot of autonomy. One of the ways you get all that is that the people working on it is if the people working on it are the de facto experts. But it could take five to six months to release, integrating would take a lot of time."*

Once it was decided to start integrating the code all the developers had been working on separately, several integration issues would arise and new bugs would be discovered. This was also the case for running their software on different platforms. Before release time, bugs specific to platforms that the original development was not performed in would be discovered and sometimes some features would have to go back to the design phase. Most of the work was contract funded and specific to a particular funding agency's needs. So the team would sometimes be supporting features no one was using anymore.

As the code base grew in size, the team started using better tools, like version control and build systems, to manage their software, as well as incorporating practices like code walkthroughs and reviews. Hal eventually transitioned out of the software development role to a research oriented role.

Philip, an experienced team member, was asked to lead the team. He eventually brought in a radically different process: Agile with Scrum for the development team. Alex recollects that *“it was many years’ worth of learning”* to get fully engaged in Agile.

Alex found that *“the way that we do in Agile here had helped tremendously . . . especially from a management side, of being able to see when you are ready or getting a true state of the application.”* Overall, Alex finds that Agile was successfully adopted and served its purpose of being able to efficiently manage and produce code, but there was a tradeoff: *“We kept the ability to say how are we going to do it, not whether it is done.”*

From coder to mentor. Over time, Alex has become one of the few people, along with Leon and **Quinn** who know the code base almost completely and have been involved in most major design decisions. Leon started working on other projects and has been away from the traditional development team work in the code base for a few years.

Alex realizes that he tends to be the most experienced person on the most teams, and a lot of his time is spent helping others. He gets less time than before to do his own share of the programming work. He has become a silo of knowledge, perhaps to his own disadvantage. *“I used to complain to Philip that I don’t get time to do my job. Because I defined my job as writing code. It took me three to four years. I realized that my job is not exactly writing code but in getting the team writing that code, testing and development done.”* Over several discussions and reviews, Alex realized, with guidance from Philip, that his role is not just to work on his own code anymore, it is to enable the team to produce good design and quality code.

As Alex gets comfortable with his newly realized role, he talks about changing his strategy for task management. *“I have evolved a pattern that if we are working on a scrum team with multiple people on the same story, unless necessary, that I will not take up a task, if I cannot do it in a timely manner, it will become a blocker or dependence for others.”*

The new strategy is to mitigate the risk for contention of resources, where the resource could even be time with someone like Alex. *“I have to have a plan in place for what we are going to do for this story, figure out what the sub tasks are and try not to take the ones that, if this one doesn’t get done, then it will prevent other tasks from being done.”* Following his own strategy for resource contention mitigation, sometimes Alex has to have the greater vision and intentionally choose to not do something he enjoys. He sees his responsibility *“to kind of be cognizant of what others are doing and even if it is something I would enjoy doing, back away from doing it.”*

Discussion: We notice that even though Alex did not originate from a software engineering discipline, he used his mechanical engineering and programming knowledge to enter this particular software development Community of Practice. He received some guidance from the existing developers but largely had to figure things out on his own.

He often worked on very large areas of the code and functionality by himself. He gained his expert status with a lot of work over a long time. This was a phase of participation through reification, where being offered a full time position, being trusted to work independently on high impact projects defined his role and interaction. He did work with other soft-

ware developers, but in a limited and sporadic capacity.

He continued to stay constantly involved in most areas of the code and participate in most major code and functionality decisions. He actively participated and supported in the change in process and teamwork practices. Through discussions with a mentor, he recognized his role in the team was changing from being defined largely by *reification* like produced code to being defined more by *participation* like enabling the team to make decisions and plan work better by using his expertise and experience.

5.2 Early onboarders: Divergent trajectories

Next, we examine some trajectories of later participants, joining the community and learning from the pioneers but forming their own distinct identities through participation and reification. Our two participants differ in an important respect: one entering as an intern and the other entering as a full time employee. This difference in initial placement affects their trajectories.

Onboarding. **Karoline** and **Ivan** had been working in the software development team for over two years and joined the company at about the same time. Ivan had extensive and varied experience in the software industry and joined the team as a full time software developer. Karoline had relatively less industry experience and joined the team initially as a software development intern.

Alex reflects on how the internship model had changed due to introduction of process. *“Now how the internship works is that just dropping in and out is not effective, there is so much more communication.”* Interns were now expected to work the same amount of hours as full time employees would.

In their initial days at the company, Karoline and Ivan got their information for the job from the pioneers. When they joined, there was one software development team and most of the onboarding process was to get dedicated time from the experts to learn the ropes and the history of the software.

Intern on the spot. Karoline in her initial position as an intern was placed in a common open area which is surrounded by the more private offices. She describes the area as a “bullpen”, where her computer screen was fairly visible to everyone in the team. Occasionally, more senior team members would come and sit next to them and give them a task to do. Then the senior member may sit on the computer next to them for the day. *“Alex would come up and sit right next to you . . . it was terrifying . . . I mean initially I didn’t know what to do and I would Google everything I didn’t know and they are right next to me, and I felt like that was quite awkward.”*

Karoline reports that these interactions were both helpful and nerve-racking, as she would feel like very visible to everyone, almost watched. She would be embarrassed that everyone could see her possibly fumble and attempt to figure out the jargon and the task assigned to her. Her comment is particularly interesting in light of another reflection about Alex: *“He is a calming influence, anytime he speaks, everybody listens . . . everything goes very smoothly when he is involved.”* Clearly it was not Alex himself but the presence of a senior member in this open space, that was the source of the discomfort. However, she says that sitting and working with different members of the team was extremely helpful and she learned a lot about the code base from the different pioneers as they all offered a different perspective based

on their expertise. She also felt embarrassed to ask many questions, not knowing if what she was asking was common knowledge or something not many others would know.

Karoline reflects on her experience of Alex's critiquing style. During a code or design review, Alex consciously prefers to pose potentially failing scenarios as questions for whether the current solution would account for it, instead of saying outright what is wrong. Karoline finds that Alex tends to authoritatively question code choices which make the less experienced programmer feel like they made the wrong code choice. The less experienced programmers can feel put on the spot. In contrast, Leon, who is just as experienced with the code base, has a different reviewing style which is more casual. He tends to say, "I think this is good, but I would suggest substituting this class for that here". Karoline and Natalie appreciate all they learn from Alex's technique but prefer Leon's style as it feels more casual and conversational.

Reification: moving out of the bullpen. Karoline getting her own office space, and not having to work in the common bullpen area made her feel accepted into the community. Karoline reflects, *"What's fun about it is when I got an office and I still got an office with Alex, but I was really like .. with my computer."* Karoline also shared that being made in charge of the maintenance team where she was responsible for onboarding other newcomers made her feel accepted, appreciated and more confident in her abilities. Karoline says, *"it definitely made me feel responsible. I didn't have someone else to defer to. It's scary at first for sure."*

Knowledge transfer: working with the pioneers. In contrast, Ivan as a full time employee was given his designated shared office space. He recollects spending long sessions with the senior programmers learning in depth about a topic about the product or code base at a time, along with the historical rationale for making decisions. He said he found those sessions very instructive and helpful.

Ivan describes what most knowledge transfer was like when he joined, *"The general feel was that Philip, at least, had a more time for one on one interaction. Alex had time for one on one interaction."* and what aspects of his interaction with the pioneers helped him. *"Initially I felt that being able to ask Philip more general and esoteric questions about the application — like the history of the components, the trajectory of development in the past, I felt that was useful for my perspective stepping in."*

Participation: feeling needed. Ivan describes how an external consultant who engaged the team in different activities, about a year after Ivan started working at the company, made him feel engaged with the team members. Ivan says, *"there is comfortability among the workers that you get from banter and pleasantries and personal conversation."* He adds that feeling needed professionally helped him feel like a part of the team, *"there is also your sense of being on the team that you get from professional interactions like going to somebody for expertise or people coming to you with questions builds that sense of like, oh, I am needed or this is who I go to or I go to this person for these questions ... I guess just knowledge of resources — human resources and brain trust resources and feeling once in a while like you can provide some value to someone else as well, really makes you feel like part of a team."*

Negotiating a common process. An interesting case

of negotiating meaning involves our two early onboarders Karoline and Ivan. Karoline and Ivan sometimes disagree on process and communication expectations, but through the retrospective and a third party mediator, they are able to come to a better understanding of their differences in expectations.

Karoline's interest in process began when Philip offered her an opportunity to participate in a high level discussion meeting about software process, a meeting she did not think she would normally be invited to. This made her curious about software process, and she read up on it in preparation for the meeting. She found that being included in that discussion has made her an even stronger proponent of software process than before.

Ivan sometimes enjoys problem solving on different tasks at the same time. Karoline, who is a software process proponent, is often of the stand that team members should typically finish and commit their current work or current task before moving on to the next task, unless there are obstacles or external delays in the current task, allowing others in the team to take up the next task when they are free and avoid bottlenecks.

In the past when working together, Karoline and Ivan have had disagreements about task management within the team. Through discussions and even third party mediation, they have come to understand their differences in expectations better. Karoline reports that since Alex and Calvin have been working on the team with Karoline and Ivan, that Alex and Calvin frequently work together with Ivan to ensure obstacles to task completion are collaboratively handled, mitigating Karoline's stress over task management.

Discussion: We notice how what helped Karoline assimilate and feel accepted in the community was formal forms of reification, like being offered a full time position and being made in charge of the maintenance team where newcomers would be onboarded or being given an office space. This may be due to the fact that Karoline entered the community with less industry experience than Ivan in a temporary internship role and was immediately surrounded by very knowledgeable and confident pioneers. In contrast, Ivan entered the community fairly confident of his abilities from his vast industry experience prior to joining this company and entered as a more permanent full time employee. So he did not need any more formal reification to feel assimilated. He felt more a part of the team from the informal participation and banter in the team and from being asked questions professionally and feeling needed.

5.3 Onboarding: the next generation

Natalie, Nathan and Casey joined the company a few years after Karoline and Ivan. At this time, the software development department had undergone a significant expansion. There were now four development teams working across two primary product lines. There was now a dedicated Scrum Master and dedicated test and documentation resources.

Initial placement. Nathan joined as a full time employee and Natalie and Casey joined as software development interns. Nathan and Natalie joined the maintenance team that follows Kanban. Casey joined one of the development teams that follows Scrum and was located in an adjacent room. This room seated all the other developers, test and documentation team members along with the

Scrum Master and Product Owner. This batch of newcomers got their knowledge from a mix of “secondary sources” — comparatively newer people like Ivan, Karoline and **Davin** teaching newly hired people in a more impromptu fashion, along with the pioneer Quinn.

Reification: Initial commit. One ritual installed by Philip is to ask the newcomers to commit code and file a bug report within the first few days of joining the software development team. This act of reification shows them how to participate and they realize that even as early as the onboarding process, they are participants and contributors instead of just observers. Philip also recommends that the newcomers take time everyday to go through the product tutorials to understand how the product works.

Secondary source mentoring. It was originally planned that there would be two experienced members, Davin and **Aurora** when the two new members join, and Aurora would be responsible for onboarding the team. However, Aurora could not be there for the first day that the interns were joining and Davin was suddenly responsible for onboarding the two new employees in the maintenance team. Davin had been working at the company himself for a little over a year, including his internship experience.

As part of the role based onboarding, Davin, who was left to creatively share knowledge in the absence of Aurora, gave Natalie and Nathan the option of either looking at a new bug themselves or watch over his shoulder while he works on a bug. Natalie and Nathan opted to look over Davin’s shoulder while he worked on a bug.

Natalie and Nathan worked together, either looking over Davin’s shoulder while he worked, thinking aloud or working on a separate bug. After Natalie and Nathan spent some time investigating, their bug, Davin sat with them, looking over their shoulders and guiding them as they continued their investigation. Davin then discussed options for a solution and gave Natalie and Nathan a lot of help implementing the solution. In the process of working through that first bug, Natalie and Nathan came across their first bug as well. They spoke to the tester about it and she confirmed that it was an unreported bug and gave them advice on how to file a bug report.

Typically, Natalie and Nathan would pair to work on different bugs, occasionally also pairing with Davin. From time to time, Natalie and Nathan would ask questions of other specific development team members that had more knowledge of that area of code and occasionally they would talk to customer proxies and testers where the bug behavior was not clear. In this way Natalie and Nathan had limited and sporadic exposure to more experienced developers and pioneers, but did not work with any for an extended period of time.

Ivan reflects on how the onboarding process changed from how he experienced it *“I think as the team grew, instead of spreading thin, I think that a lot of the interaction didn’t occur at all and hopefully that hasn’t been too detrimental ... Hopefully some of the less experienced engineers, that was Philip’s intention and plan, pick up that slack. The new hires and interns tend to rely more on everybody as opposed to a few. And so, we(secondary sources) are not as familiar with providing that kind of direction, nor do we necessarily know all the answers to questions that an experienced engineer would.”*

The three newcomers recognized that one of the disad-

vantages of not having access to the pioneers was that the overview of the product, the code architecture and organization and the historical context and rationale behind major design decisions was not obvious for a long time. Casey reflected that the lack of detailed documentation where one can go and learn some of these things initially hampered his ability to confidently understand the code and its organization.

Participation: Working with a pioneer. Casey reports feeling very lost and stressed the first couple of days at the company. He felt overwhelmed with the jargon and trying to understand new tools and systems all at the same time. Casey joined the Scrum team and worked with senior programmer Quinn on a small story for a while when he first started on the team. Quinn would be on the phone and screen share with Casey and would walk him through the problem and solution for the new feature. He would also work on the code in front of Casey so that Casey could follow along Quinn’s work. Casey found the extended one on one sessions extremely helpful and started to feel like he knew how to start working with the code base more easily.

Team reorganization: After about a month and a half from the time the newcomers joined, the teams were reorganized. The maintenance team following Kanban was renamed to be the pre-release team and many members of two of the Scrum development teams were placed in the pre-release team, to prepare for a major version release. The pre-release team became larger with about six to ten members at any given time.

As part of this team, Natalie and Nathan got the opportunity to work more closely with other developers more regularly. Here they had access to one pioneer Alex and sometimes another highly experienced member Quinn. And other secondary sources like Karoline and Ivan. Casey who also joined the pre-release team got to experience what it is like to work in vastly different areas of the code and to work on smaller pieces of work, where most bugs were smaller than user stories worked upon in the Scrum model.

After almost two months of working in that model, the teams got reorganized again. A new Scrum development team was formed, that included Alex, Karoline, Ivan, Natalie, Nathan and Casey. At a later time, Calvin also joined this team later as Nathan moved on to work on a different team. In this model, Natalie and Nathan got to work on larger stories for extended periods of time, which was a different style and pace of work most of the Kanban work.

Participation: From maintenance to development. Natalie reports that she enjoyed the pace of the Scrum work more than the Kanban work, as she got to further her knowledge and really understand something and work more closely with more people in the Scrum format. She felt that the larger but close knit team in the Scrum format made her feel more like a part of a team and more confident. In the Kanban format, she felt that she got to speak to many of the developers, but in very minimal capacities and did not feel like she got to properly pair and work with anyone except Nathan. So the smaller team Kanban work felt more like not working independently in a team as opposed to working together in a team in Scrum.

Reification: From bugs to stories. Natalie also reports that working on stories which were new features or new enhancements to existing features felt more rewarding than the bug fixing mode, perhaps because in the Scrum

team, her work felt more creative and she produced a more significant volume of code for that amount of work whereas in the bug fixing mode, she may work for a whole day to find the cause of a bug and fix it with just one line. The work felt more like critiquing and correcting than creative.

Negotiating a workable Kanban. The newly formed pre-release team was discussing the rules of engagement for Kanban. The team was told how the Work In Progress (WIP) Limit for a team is typically between half the team size and a few numbers less than the team size, to allow some members to be able to pair on some of the tasks. The recommendation from the Scrum Master and the team manager were to keep a relatively small WIP limit to encourage more pairing. The team decided after some discussion that they wanted to keep the work flowing faster and would prefer a WIP limit of one to two less than their current team size. This was a case where the self-managing team, after discussion arrived at a decision that they were advised against.

Discussion: We find that for the newcomers to the community, the change in team size and organization meant that they did not always have as much free access to the pioneers and primary sources of knowledge. They had sporadic access and took longer to understand things like the reason behind certain design decisions and the historical context of the code organization, etc. The early onboarders got this information from the pioneers not so long ago. Perhaps in the rapid team expansion and segregation, some of that historical context and rationale that Ivan and Karoline found so useful was not written down to be formally passed down the generations of developers.

6. CONCLUSION

In this work, we have used the Community of Practice framework in a descriptive fashion, explaining participants' choices within the TAI organization in terms of negotiated meaning and trajectories of identity, and mindful of the history of the firm. We find Community of Practice to be a highly useful tool for deriving the essence of the onboarding and mentoring process. We have explored through a mix of participant and observer accounts how background and the work context can help determine what forms of reification and participation would be most effective in promoting integration into the community. Through our interviews with a range of community members, we have seen how knowledge transfer practices have changed to allow different members to progress their role and trajectory. Our *emic*, or "insider", perspective has the advantage of capturing concepts in the terms of the participants themselves, rather than forcing them into established categories. In our study, we observe the lived experience of software development, augmented with knowledge of how such a software community forms and sustains itself through scope and size changes.

This grounded study serves as a generator of research questions for further investigations. Through our open coding approach [13], we arrived at several questions we intend to pursue in a more focused fashion:

- **Onboarding and anxiety.** There is clearly value in bringing newcomers in quickly through participation and reification, yet there are risks of putting these participants "on the spot" in potentially detrimental ways. What onboarding practices can best bring new employees into the community quickly while minimiz-

ing unproductive anxiety?

- **Pioneers as rationale sources.** One tremendous value of working with pioneer mentors was the access to *rationale* [4]: not only the *what* but the *why* behind the TAI codebase. As mentoring evolves away from an exclusively pioneer duty, pioneers and rationale must be decoupled. How can knowledge of design choices — what was done, and what was *not* done — be transferred accurately and effectively?
- **Mentoring the mentors.** Our study revealed a range of mentoring styles, which is unsurprising given the evolving nature of mentoring at TAI. Are certain mentoring styles particularly well suited for certain personalities, or certain stages of the onboarding process? To what degree can an onboarding program for mentors help in this regard?

Future work will draw from other workplace scenarios, including an ongoing study of mentoring in open-source projects [8]. Through a series of such studies, we will identify patterns of team communication and interaction that foster effective mentoring strategies.

7. ACKNOWLEDGMENTS

We express sincere thanks to the TAI management and its team of developers, particularly our interviewees who provided the basis of this work.

8. REFERENCES

- [1] A. Begel. Help, I need somebody! In *Supporting the Social Side of Large Scale Software Development*, page 59. Microsoft Research, 2006.
- [2] A. Begel and B. Simon. Novice software developers, all over again. In *Proceedings of the Fourth international Workshop on Computing Education Research*, pages 3–14. ACM, 2008.
- [3] C. Buell. Models of mentoring in communication. *Communication Education*, 53(1), 2004.
- [4] J. Burge, J. Carroll, R. McCall, and I. Mistrík. *Rationale-Based Software Engineering*. Springer, 2008.
- [5] T. Hall, S. Beecham, J. Verner, and D. Wilson. The impact of staff turnover on software projects: the importance of understanding what makes software practitioners tick. In *Proceedings of the 2008 ACM SIGMIS CPR conference on Computer Personnel Doctoral Consortium*, pages 30–39. ACM, 2008.
- [6] P. J. Kalbfleisch. Communicating in mentoring relationships: A theory for enactment. *Communication Theory*, 12(1):63–69, 2002.
- [7] H. Kniberg and A. Ivarsson. Scaling agile @ spotify with tribes, squads, chapters & guilds. <http://blog.crisp.se/2012/11/14/henrikkniberg/scaling-agile-at-spotify>, 2012.
- [8] S. Kumar and C. Wallace. Communication strategies for mentoring in software development projects. In *Proceedings of the 7th International Workshop on Cooperative and Human Aspects of Software Engineering*, pages 111–114. ACM, 2014.

- [9] J. Lave and E. Wenger. Legitimate peripheral participation in communities of practice. *Supporting lifelong learning*, 1:111–126, 2002.
- [10] National Society of Professional Engineers. *Mentoring Guide for Small, Medium, and Large Firms*, 2002.
- [11] M. Paasivaara and C. Lassenius. Communities of practice in a large distributed agile software development organization — Case Ericsson. *Information and Software Technology*, 56(12):1556–1577, 2014.
- [12] M. Serrano Zanetti. The co-evolution of socio-technical structures in sustainable software development: Lessons from the open source software communities. In *Proceedings of the 2012 International Conference on Software Engineering*, pages 1587–1590. IEEE Press, 2012.
- [13] A. Strauss and J. M. Corbin. *Basics of qualitative research: Grounded theory procedures and techniques*. Sage, 1990.
- [14] E. Wenger. *Communities of practice: Learning, meaning, and identity*. Cambridge university press, 1998.
- [15] Y. Yu, A. Benlian, and T. Hess. An empirical study of volunteer members’ perceived turnover in open source software projects. In *Hawaii International Conference on System Science (HICSS)*, pages 3396–3405. IEEE, 2012.